NebulaTwin Pro — Complete Documentation

Cloud-Based Digital Twin SaaS Platform Version: 0.4.0 (Intelligence, Collaboration & Safety) | Last Updated: May 2026

Table of Contents

1. Project Overview

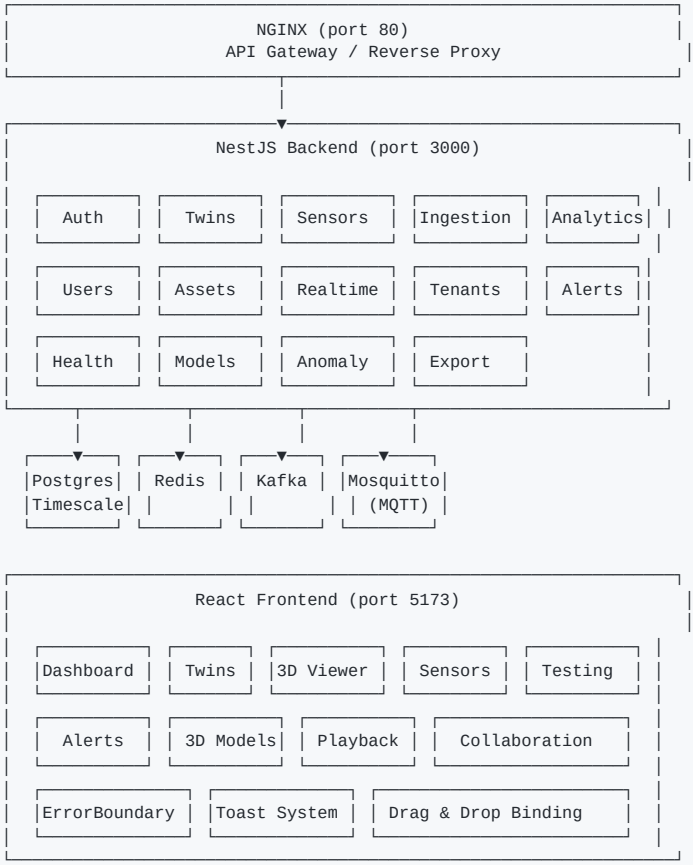
NebulaTwin Pro is a full-stack, cloud-based Digital Twin SaaS platform. It allows industrial users to:

- Create and manage **digital twins** of physical assets (factories, production lines, machines, components)
- Upload and interact with **3D models** in the browser, with **versioning and rollback (v0.4.0)**
- Attach **sensors** to machine parts via click-to-bind or **drag-and-drop** onto 3D meshes **(v0.4.0)**
- **Simulate sensor data** with configurable patterns (sine, random, linear, constant)
- **Manually override** sensor values for testing
- Receive **real-time alerts** when sensor values breach configurable thresholds, with **cooldown and hysteresis (v0.4.0)**
- **AI anomaly detection** — automatic statistical anomaly scoring on every data point **(v0.4.0)**
- Visualize data in **real-time dashboards** with charts and 3D color-coded models
- **Playback sensor history** through the 3D viewer with time-range selection **(v0.4.0)**
- **Real-time collaboration** — see who's editing, broadcast mesh selection **(v0.4.0)**
- **Export & share** — download sensor CSV/JSON, create time-limited public share links **(v0.4.0)**
- Ingest sensor data via **HTTP** and **MQTT** protocols, with **per-sensor schema validation (v0.4.0)**
- Query **time-series analytics** (history, latest, aggregated buckets)

The platform is **multi-tenant**, supports **RBAC** (Role-Based Access Control), and is designed for horizontal scalability.

---

2. Architecture



3. Tech Stack

Backend

Technology	Purpose	Version
NestJS	Application framework	^10.3.0
TypeScript	Language	^5.3.3
Prisma	ORM / database client	^5.8.0
PostgreSQL	Relational database	16
TimescaleDB	Time-series extension	latest-pg16
Redis	Caching, pub/sub, event bus	7-alpine
Apache Kafka	Event streaming / message broker	3.7.0
Mosquitto	MQTT broker for IoT ingestion	2
Socket.IO	WebSocket real-time gateway	^10.3.0
Passport	Authentication (JWT + Google OAuth)	^0.7.0
Swagger	API documentation	^7.2.0
Helmet	Security headers	^7.1.0

Frontend

Technology	Purpose	Version
React	UI framework	^19.2.5
TypeScript	Language	~6.0.2
Vite	Build tool	^8.0.10
Tailwind CSS	Styling	^4.2.4
Zustand	State management	^5.0.12
TanStack Query	Server state / caching	^5.100.6
Three.js	3D rendering engine	^0.184.0
@react-three/fiber	React Three.js bindings	^9.6.1
@react-three/drei	Three.js helpers / controls	^10.7.7
Recharts	Charts and data visualization	^3.8.1
Socket.IO Client	WebSocket client	^4.8.3
Axios	HTTP client	^1.15.2
Lucide React	Icons	^1.14.0
React Router	Client-side routing	^7.14.2

Infrastructure

Technology	Purpose
Docker Compose	Container orchestration
NGINX	API gateway, reverse proxy, rate limiting
Node.js 20	Runtime (Alpine image)

4. Project Structure

Root

```
nebula/
├── src/                          # Backend source code (NestJS)
├── frontend/                     # Frontend source code (React)
├── prisma/                       # Database schema & migrations
│   ├── schema.prisma             # Prisma schema definition
│   ├── seed.ts                   # Database seed script
│   └── migrations/               # SQL migrations
├── docker/                       # Docker configuration files
│   ├── nginx/nginx.conf          # NGINX reverse proxy config
│   └── mosquitto/mosquitto.conf  # MQTT broker config
├── docker-compose.yml            # Full infrastructure definition
├── Dockerfile                    # Multi-stage production build
├── .env.example                  # Environment variable template
├── .env                          # Local environment (gitignored)
├── package.json                  # Backend dependencies & scripts
├── tsconfig.json                 # TypeScript configuration
└── nest-cli.json                 # NestJS CLI configuration
```

Backend (`src/`)

```
src/
├── main.ts                      # Application entry point & bootstrap
├── app.module.ts                # Root module (imports all feature modules)
│
├── database/                    # Database layer
│   ├── database.module.ts       # Database module (Prisma + TimescaleDB)
│   ├── prisma.service.ts        # Prisma client lifecycle management
│   └── timescale.service.ts     # TimescaleDB raw queries (hypertable)
│
├── common/                      # Shared utilities
│   ├── decorators/
│   │   ├── current-user.decorator.ts   # @CurrentUser() param decorator
│   │   └── roles.decorator.ts          # @Roles() method decorator
│   ├── guards/
│   │   ├── roles.guard.ts       # RBAC role guard
│   │   └── tenant.guard.ts      # Tenant isolation guard
│   ├── redis/
│   │   ├── redis.module.ts      # Redis module (ioredis)
│   │   └── redis.service.ts     # Redis get/set/pub/sub service
│   └── event-bus/
│       ├── event-bus.module.ts  # Kafka event bus module
│       └── event-bus.service.ts  # Kafka producer/consumer service
│
└── modules/                     # Feature modules
    ├── auth/                    # Authentication & authorization
    │   ├── auth.module.ts
    │   ├── auth.controller.ts   # Login, register, refresh, logout, Google OAuth
    │   ├── auth.service.ts      # JWT generation, password hashing, Google validation
    │   ├── dto/
    │   │   ├── login.dto.ts     # { email, password }
    │   │   └── register.dto.ts  # { email, password, name, tenantId?, role? }
    │   ├── guards/
    │   │   └── jwt-auth.guard.ts # JWT authentication guard
    │   └── strategies/
    │       ├── jwt.strategy.ts  # JWT Passport strategy
    │       └── google.strategy.ts # Google OAuth Passport strategy
    │
    ├── users/                   # User management
    │   ├── users.module.ts
    │   ├── users.controller.ts  # CRUD, role assignment
    │   └── users.service.ts     # User queries, Google linking
    │
    ├── tenants/                 # Multi-tenant management
    │   ├── tenants.module.ts
    │   ├── tenants.controller.ts # CRUD for tenants
    │   └── tenants.service.ts
    │
    ├── twins/                   # Digital Twin management
    │   ├── twins.module.ts
    │   ├── twins.controller.ts  # CRUD with nested includes
    │   └── twins.service.ts
    │
    ├── assets/                  # Asset hierarchy management
    │   ├── assets.module.ts
    │   ├── assets.controller.ts  # CRUD, root assets, tree queries
    │   └── assets.service.ts
    │
    ├── sensors/                 # Sensor management & control
    │   ├── sensors.module.ts
    │   ├── sensors.controller.ts # CRUD, override, stream, bind/unbind
    │   ├── sensors.service.ts    # Sensor business logic
    │   ├── sensor-stream.service.ts # Simulated data stream engine
    │   └── dto/
    │       ├── create-sensor.dto.ts   # { name, type, unit, assetId? }
    │       ├── override-sensor.dto.ts # { mode, value }
    │       └── start-stream.dto.ts    # { pattern, interval_ms, min, max }
    │
    ├── ingestion/               # Data ingestion (HTTP + MQTT)
    │   ├── ingestion.module.ts
    │   ├── ingestion.controller.ts    # POST /ingest, POST /ingest/batch
    │   ├── ingestion.service.ts       # Validate & store data points
    │   └── mqtt-ingestion.service.ts  # MQTT subscriber → ingestion pipeline
    │
    ├── realtime/                # WebSocket real-time gateway + collaboration (v0.4.0)
    │   ├── realtime.module.ts
    │   ├── realtime.gateway.ts   # Socket.IO gateway (subscribe/broadcast + collaboration presence/selection)
    │   └── realtime.service.ts   # Redis pub/sub → WebSocket bridge
    │
    ├── analytics/               # Time-series analytics
    │   ├── analytics.module.ts
    │   ├── analytics.controller.ts    # History, latest, aggregated endpoints
```

```
│   └── analytics.service.ts       # TimescaleDB queries with Redis caching
│
├── models/                  # 3D Model management (v0.3.0, versioning+soft-delete v0.4.0)
│   ├── models.module.ts           # Module with Multer file upload config
│   ├── models.controller.ts       # Upload, CRUD, versioning, rollback, soft delete, restore
│   ├── models.service.ts          # File storage, mesh parsing, versioning, soft delete
│   └── dto/
│       ├── create-model.dto.ts    # { twinId, name?, description? }
│       └── update-model.dto.ts    # { name?, description? }
│
├── anomaly/                 # AI Anomaly Detection (v0.4.0)
│   ├── anomaly.module.ts          # Exports AnomalyDetectionService
│   └── anomaly-detection.service.ts # Rolling z-score + spike detection
│
└── export/                  # Export & Sharing (v0.4.0)
    ├── export.module.ts           # ExportModule
    ├── export.controller.ts       # CSV/JSON export, share links, public access
    └── export.service.ts          # Data export + token-based share link management
```

Frontend (`frontend/src/`)

```
frontend/src/
├── main.tsx                    # React entry point
├── App.tsx                     # Router + providers setup
├── index.css                   # Tailwind CSS + custom theme
│
├── types/
│   └── index.ts                # TypeScript interfaces (User, Twin, Sensor, etc.)
│
├── services/
│   ├── api.ts                  # Axios client + all API functions
│   └── websocket.ts            # Socket.IO client + event subscription
│
├── utils/
│   ├── cn.ts                   # Tailwind class merge utility
│   └── rbac.ts                 # useRole() hook for frontend RBAC (v0.3.0)
│
├── store/
│   ├── authStore.ts            # Auth state (login, logout, tokens)
│   ├── twinStore.ts            # Twins + assets state
│   ├── sensorStore.ts          # Sensors + realtime values state
│   └── viewerStore.ts          # 3D viewer selection state (extended v0.3.0)
│
├── components/
│   ├── ui/
│   │   ├── Button.tsx          # Styled button (variants: primary/secondary/outline/ghost/destructive)
│   │   ├── Card.tsx            # Card + CardHeader + CardTitle + CardContent
│   │   ├── Input.tsx           # Styled input field
│   │   ├── Badge.tsx           # Status badge (default/success/warning/danger)
│   │   ├── ConfirmDialog.tsx   # Reusable destructive action confirmation modal (v0.3.0)
│   │   ├── Skeleton.tsx        # Loading skeletons: Skeleton, CardSkeleton, TableSkeleton, ListSkeleton (v0.4.0)
│   │   └── EmptyState.tsx      # Empty state with icon, title, description, optional action (v0.4.0)
│   │
│   ├── layout/
│   │   ├── AppLayout.tsx       # Protected layout with sidebar + outlet
│   │   └── Sidebar.tsx         # Navigation sidebar with icons
│   ├── 3d/
│   │   ├── SceneViewer.tsx     # Three.js canvas + SceneDragDrop wrapper + SceneCapture (updated v0.4.0)
│   │   ├── DemoFactory.tsx     # Interactive 3D factory model
│   │   ├── UploadedModel.tsx   # Dynamic 3D model loader (GLTFLoader/OBJLoader) (v0.3.0)
│   │   └── SensorDragOverlay.tsx # SceneDragDrop (raycast drop zone) + DraggableSensorChip (v0.4.0)
│   ├── sensors/
│   │   └── SensorBindingPanel.tsx # Bind/unbind/drag sensors to model parts (updated v0.4.0)
│   ├── models/
│   │   └── VersionSelector.tsx   # Model version history dropdown + rollback (v0.4.0)
│   ├── playback/
│   │   └── PlaybackControls.tsx  # Sensor history playback with time range + speed control (v0.4.0)
│   └── collaboration/
│       └── PresenceIndicator.tsx # Active collaborator avatars (v0.4.0)
│
└── pages/
    ├── auth/
    │   └── LoginPage.tsx       # Login form + Google OAuth button
    ├── dashboard/
    │   └── DashboardPage.tsx   # KPI cards + charts + alerts + live values
    ├── twins/
    │   └── TwinsPage.tsx       # Twin list + create/edit/delete + asset tree (updated v0.3.0)
    ├── viewer/
    │   └── ViewerPage.tsx      # 3D viewer + version selector + playback + collaboration + drag-drop (updated v0.4.0)
    ├── models/
    │   └── ModelsPage.tsx      # Upload, versioning, soft delete, restore, version selector (updated v0.4.0)
    ├── alerts/
    │   └── AlertsPage.tsx      # Alert list with INFO/WARNING/CRITICAL badges, improved empty state (updated v0.4.0)
    └── sensors/
        ├── SensorsPage.tsx     # Sensor grid + create/edit/delete (updated v0.3.0)
        └── SensorTestingPage.tsx # Full sensor control panel
```

5. Backend (NestJS)

5.1 Modules

Module	Description
`AppModule`	Root module, imports all feature modules, configures throttling and scheduling
`DatabaseModule`	Provides `PrismaService` and `TimescaleService` globally
`RedisModule`	Provides `RedisService` (ioredis client) globally
`EventBusModule`	Kafka producer/consumer for event streaming
`AuthModule`	JWT auth, Google OAuth, login/register/refresh/logout
`UsersModule`	User CRUD, role management, Google ID linking
`TenantsModule`	Tenant CRUD for multi-tenancy
`TwinsModule`	Digital twin CRUD with nested asset/model includes
`AssetsModule`	Hierarchical asset CRUD (factory → line → machine → component)
`SensorsModule`	Sensor CRUD, manual override, stream simulation, model binding, anomaly detection integration, device validation **(updated v0.4.0)**
`IngestionModule`	HTTP + MQTT data ingestion into TimescaleDB, rate limiting, metrics
`RealtimeModule`	Socket.IO WebSocket gateway with throttled broadcast + collaboration presence/selection **(updated v0.4.0)**
`AnalyticsModule`	Time-series queries (history, latest, aggregated with time_bucket)
`AlertsModule`	Alert CRUD, acknowledgment, threshold-based alert creation
`HealthModule`	Health check endpoints (`/health` , `/health/live` , `/health/ready`)
`ModelsModule`	3D model upload, CRUD, mesh parsing, versioning, rollback, soft delete, restore **(updated v0.4.0)**
`AnomalyModule`	AI anomaly detection — rolling z-score + spike analysis, injectable into SensorsModule **(v0.4.0)**
`ExportModule`	CSV/JSON data export, token-based public share links **(v0.4.0)**

5.2 Database Schema

PostgreSQL + TimescaleDB

Prisma-Managed Tables

Table	Description	Key Fields
`tenants`	Multi-tenant organizations	id, name
`users`	User accounts	id, email, passwordHash, googleId, role, tenantId
`digital_twins`	Digital twin definitions	id, name, description, tenantId
`assets`	Hierarchical asset tree	id, name, type (FACTORY/LINE/MACHINE/COMPONENT), twinId, parentId
`models_3d`	3D model files	id, name, description, fileUrl, format (ModelFormat enum), sizeBytes, meshStructure, twinId, **tenantId**, **version**, **isLatest**, **parentModelId**, **deletedAt (v0.4.0)**
`model_parts`	Individual mesh parts of 3D models	id, name, modelId, metadata
`sensors`	Sensor definitions + control state	id, name, type, unit, mode, manualValue, stream*, assetId, modelPartId, tenantId, **alertCooldownMs**, **alertHysteresis**, **validationSchema (v0.4.0)**
`audit_logs`	Activity audit trail	id, action, entity, entityId, details, userId, tenantId

TimescaleDB Hypertable (raw SQL)

Table	Description	Columns
`sensor_data`	Time-series sensor readings	time (TIMESTAMPTZ), sensor_id (TEXT), value (DOUBLE PRECISION), metadata (JSONB)

- Partitioned by `time` using `create_hypertable()`
- Indexed on `(sensor_id, time DESC)`
- Supports `time_bucket()` aggregation queries

Enums

Enum	Values
`Role`	ADMIN, MANAGER, OPERATOR, VIEWER
`SensorMode`	REAL, MANUAL, **STREAM**
`AssetType`	FACTORY, LINE, MACHINE, COMPONENT
`StreamPattern`	CONSTANT, LINEAR_INCREASE, LINEAR_DECREASE, SINE, RANDOM
`ModelFormat`	GLTF, GLB, OBJ **(v0.4.0)**
`AlertSeverity`	**INFO (v0.4.0)**, WARNING, CRITICAL

Alert Model (new in v0.2.0)

Field	Type	Description
`id`	UUID	Primary key
`severity`	AlertSeverity	INFO, WARNING, or CRITICAL
`message`	String	Human-readable alert description
`value`	Float	The sensor value that triggered the alert
`acknowledged`	Boolean	Whether the alert has been acknowledged
`sensorId`	FK → Sensor	Sensor that triggered the alert
`tenantId`	FK → Tenant	Tenant isolation
`createdAt`	DateTime	Timestamp

Sensor Threshold Fields (new in v0.2.0)

Field	Type	Description
`alertMinThreshold`	Float?	Minimum threshold — values below trigger alert
`alertMaxThreshold`	Float?	Maximum threshold — values above trigger alert

Sensor Advanced Fields (new in v0.4.0)

Field	Type	Description
`alertCooldownMs`	Int?	Minimum milliseconds between alerts for this sensor (default 30000)
`alertHysteresis`	Float?	Value must exceed threshold by this amount before triggering
`validationSchema`	Json?	Per-sensor payload validation: `{ minValue?, maxValue?, valueType?, requiredMetadata? }`

Model Versioning Fields (new in v0.4.0)

Field	Type	Description
`version`	Int	Version number (starts at 1, auto-increments)
`isLatest`	Boolean	Whether this is the latest version in the lineage
`parentModelId`	String?	FK → Model3D, points to root model of this version chain
`deletedAt`	DateTime?	Soft delete timestamp (null = active, non-null = deleted)

5.3 API Endpoints

Base URL: `/api/v1`

Authentication

Method	Path	Auth	Description
POST	`/auth/register`	No	Register new user
POST	`/auth/login`	No	Login with email/password → returns JWT tokens
POST	`/auth/refresh`	No	Refresh access token
POST	`/auth/logout`	JWT	Invalidate refresh token
GET	`/auth/google`	No	Initiate Google OAuth flow
GET	`/auth/google/callback`	No	Google OAuth callback

Login Response:

```
{
  "user": { "id": "uuid", "email": "...", "name": "...", "role": "ADMIN" },
  "accessToken": "eyJ...",
  "refreshToken": "eyJ..."
}
```

Tenants

Method	Path	Auth	Roles	Description
POST	`/tenants`	JWT	ADMIN	Create tenant
GET	`/tenants`	JWT	ADMIN	List all tenants
GET	`/tenants/:id`	JWT	ADMIN	Get tenant by ID
PATCH	`/tenants/:id`	JWT	ADMIN	Update tenant
DELETE	`/tenants/:id`	JWT	ADMIN	Delete tenant

Users

Method	Path	Auth	Roles	Description
GET	`/users`	JWT	ADMIN, MANAGER	List users
GET	`/users/me`	JWT	Any	Get current user profile
PATCH	`/users/:id/role`	JWT	ADMIN	Change user role

Digital Twins

Method	Path	Auth	Description
POST	`/twins`	JWT	Create digital twin
GET	`/twins`	JWT	List twins (with assets & models)
GET	`/twins/:id`	JWT	Get twin with full hierarchy
PATCH	`/twins/:id`	JWT	Update twin
DELETE	`/twins/:id`	JWT	Delete twin

Assets

Method	Path	Auth	Description
POST	`/assets`	JWT	Create asset
GET	`/assets`	JWT	List assets (filter by twinId)
GET	`/assets/:id`	JWT	Get single asset
GET	`/assets/roots/:twinId`	JWT	Get root assets of a twin
PATCH	`/assets/:id`	JWT	Update asset
DELETE	`/assets/:id`	JWT	Delete asset

Sensors

Method	Path	Auth	Description
POST	`/sensors`	JWT	Create sensor
GET	`/sensors`	JWT	List all sensors
GET	`/sensors/:id`	JWT	Get sensor by ID
GET	`/sensors/by-asset/:assetId`	JWT	Get sensors for an asset
PATCH	`/sensors/:id`	JWT	Update sensor
DELETE	`/sensors/:id`	JWT	Delete sensor
POST	`/sensors/:id/override`	JWT	ADMIN, MANAGER
POST	`/sensors/:id/override/clear`	JWT	ADMIN, MANAGER
POST	`/sensors/:id/stream`	JWT	ADMIN, MANAGER
POST	`/sensors/:id/stop`	JWT	ADMIN, MANAGER
GET	`/sensors/streams/active`	JWT	List active stream IDs
POST	`/sensors/:id/bind/:modelPartId`	JWT	Bind sensor to 3D model part
POST	`/sensors/:id/unbind`	JWT	Unbind sensor from model part

Override Request:

```
{ "value": 72.5 }
```

> **Note (v0.2.0):** The `mode` field is now deprecated and optional. Override always sets MANUAL mode.

Stream Request:

```
{
  "pattern": "SINE",
  "interval_ms": 1000,
  "min": 10,
  "max": 90
}
```

3D Models (v0.3.0, updated v0.4.0)

Method	Path	Auth	Roles	Description
POST	`/models`	JWT	ADMIN, MANAGER	Upload 3D model file (multipart/form-data)
GET	`/models`	JWT	Any	List models (filter by `?twinId=`, `?includeDeleted=true`) **(v0.4.0)**
GET	`/models/:id`	JWT	Any	Get model with parts and sensors
PATCH	`/models/:id`	JWT	ADMIN, MANAGER	Update model name/description
DELETE	`/models/:id`	JWT	ADMIN, MANAGER	**Soft delete** model (sets `deletedAt`) **(v0.4.0)**
GET	`/models/:id/bound-sensors`	JWT	Any	Count sensors bound to model parts
POST	`/models/:id/version`	JWT	ADMIN, MANAGER	Upload new version of model (multipart/form-data) **(v0.4.0)**
GET	`/models/:id/versions`	JWT	Any	Get version history for model lineage **(v0.4.0)**
POST	`/models/:id/rollback`	JWT	ADMIN, MANAGER	Rollback — promote this version to latest **(v0.4.0)**
POST	`/models/:id/restore`	JWT	ADMIN, MANAGER	Restore soft-deleted model **(v0.4.0)**
DELETE	`/models/:id/permanent`	JWT	ADMIN	Permanently delete model and file **(v0.4.0)**

Upload Request (multipart/form-data):

Field	Type	Required	Description
`file`	File	Yes	3D model file (.glb, .gltf, .obj)
`twinId`	String	Yes	Digital twin to associate with
`name`	String	No	Model name (defaults to filename)
`description`	String	No	Model description

Upload Response:

```
{
  "id": "uuid",
  "name": "Factory Floor",
  "fileUrl": "/uploads/models/uuid.glb",
  "format": "GLB",
  "sizeBytes": 5242880,
  "twinId": "twin-uuid",
  "tenantId": "tenant-uuid",
  "modelParts": [
    { "id": "part-uuid", "name": "Motor_Housing", "modelId": "uuid" },
    { "id": "part-uuid", "name": "Conveyor_Belt", "modelId": "uuid" }
  ]
}
```

File validation: Allowed extensions: `.glb`, `.gltf`, `.obj`. Max size: 100MB (configurable via `MAX_FILE_SIZE`).

Mesh parsing: On upload, GLTF/GLB files are parsed to extract mesh names. Each unique mesh becomes a `ModelPart` with a UUID, enabling sensor binding by `modelPartId`.

Cascade delete: Deleting a model unbinds all sensors from its parts, removes the file from disk, and deletes the model + parts from the database.

Ingestion

Method	Path	Auth	Description
POST	`/ingest`	No	Ingest single data point
POST	`/ingest/batch`	No	Ingest batch of data points
GET	`/ingest/metrics`	JWT (ADMIN, MANAGER)	Get ingestion metrics (totalIngested, totalRejected, totalRateLimited)

Single Ingest Response (v0.2.0):

```
{ "status": "accepted" }
// or
{ "status": "rejected", "reason": "sensor_in_manual_mode" }
// or
{ "status": "rejected", "reason": "rate_limited" }
```

Ingestion enforces sensor mode: If sensor is in `MANUAL` or `STREAM` mode, external ingestion is rejected.

Rate limiting: Max 20 ingestions per sensor per second (Redis-backed sliding window).

Alerts (new in v0.2.0)

Method	Path	Auth	Roles	Description
GET	`/alerts`	JWT	Any	List alerts (query: `?limit=N`)
GET	`/alerts/unacknowledged`	JWT	Any	List unacknowledged alerts
GET	`/alerts/stats`	JWT	Any	Alert statistics (total, unacknowledged, critical, warning)
GET	`/alerts/sensor/:sensorId`	JWT	Any	Alerts for a specific sensor
POST	`/alerts/:id/acknowledge`	JWT	ADMIN, MANAGER, OPERATOR	Acknowledge alert
POST	`/alerts/acknowledge-all`	JWT	ADMIN, MANAGER	Acknowledge all alerts

Health (new in v0.2.0)

Method	Path	Auth	Description
GET	`/health`	No	Full health check (DB, Redis, TimescaleDB with latency)
GET	`/health/live`	No	Liveness probe (always returns OK)
GET	`/health/ready`	No	Readiness probe (checks DB connection)

Health Response:

```
{
  "status": "healthy",
  "uptime": 48.04,
  "timestamp": "2026-04-30T23:42:48.030Z",
  "checks": {
    "database": { "status": "up", "latencyMs": 6 },
    "redis": { "status": "up", "latencyMs": 1 },
    "timescaledb": { "status": "up", "latencyMs": 3 }
  }
}
```

Single Ingest:

```
{ "sensor_id": "sensor-temp-01", "value": 23.5 }
```

Batch Ingest:

```
{
  "data": [
    { "sensor_id": "sensor-temp-01", "value": 23.5 },
    { "sensor_id": "sensor-vibr-01", "value": 4.2 }
  ]
}
```

Analytics

Method	Path	Auth	Description
GET	`/analytics/sensors/:sensorId/history`	JWT	Time-series history (params: from, to, limit)
GET	`/analytics/sensors/:sensorId/latest`	JWT	Most recent data point
GET	`/analytics/sensors/:sensorId/aggregated`	JWT	Aggregated buckets (params: from, to, interval)

Aggregated Response:

```
[
  {
    "bucket": "2026-04-30T22:00:00.000Z",
    "sensor_id": "sensor-vibr-01",
    "avg_value": 52.3,
    "min_value": 10.0,
    "max_value": 90.0,
    "count": 60
  }
]
```

MQTT Ingestion (non-HTTP)

- Topic: `sensors/{sensorId}/data`
- Payload: `{ "value": 23.5, "metadata": {} }`
- The `MqttIngestionService` subscribes to `sensors/+/data` and routes messages to the ingestion pipeline

Export & Sharing (new in v0.4.0)

Method	Path	Auth	Roles	Description
GET	`/export/sensors/:id/csv`	JWT	Any	Export sensor data as CSV (query: `?from=&to=`)
GET	`/export/twins/:id/json`	JWT	Any	Export full twin config as JSON
POST	`/export/twins/:id/share`	JWT	ADMIN, MANAGER	Create a time-limited public share link
GET	`/export/shared/:token`	No	—	Access shared twin by token (public, read-only)
DELETE	`/export/shared/:token`	JWT	ADMIN, MANAGER	Revoke a share link

Share Link Response:

```
{
  "token": "abc123...",
  "url": "/api/v1/export/shared/abc123...",
  "expiresAt": "2026-05-07T00:00:00.000Z"
}
```

5.4 Authentication & Authorization

JWT Authentication:

- Access tokens expire in 1 day (configurable via `JWT_EXPIRES_IN`)
- Refresh tokens expire in 7 days (configurable via `JWT_REFRESH_EXPIRES_IN`)
- Tokens contain: `sub` (userId), `email`, `role`, `tenantId`
- Refresh tokens are bcrypt-hashed and stored in the user record
- The `JwtAuthGuard` protects all authenticated endpoints

Google OAuth:

- Uses `passport-google-oauth20` strategy
- Automatically creates user on first Google login
- Links Google ID to existing email if account exists
- Conditionally loaded — skipped if `GOOGLE_CLIENT_ID` is not configured

RBAC (Role-Based Access Control):

- 4 roles: `ADMIN` > `MANAGER` > `OPERATOR` > `VIEWER`
- `@Roles('ADMIN', 'MANAGER')` decorator on controller methods
- `RolesGuard` checks user role against required roles
- `TenantGuard` ensures users can only access their own tenant's data
- **v0.2.0:** Override and stream control endpoints restricted to `ADMIN` and `MANAGER` only (was ADMIN/MANAGER/OPERATOR)

5.5 Real-Time System

The real-time pipeline:

```
Sensor Data → TimescaleDB INSERT
           → Redis PUBLISH (channel: sensor:{sensorId}:data)
           → RealtimeService listens to Redis
           → Broadcasts via Socket.IO to subscribed clients
```

WebSocket Namespace: `/realtime`

Client Events (emit):

Event	Payload	Description
`subscribe:tenant`	`{ tenantId }`	Join tenant room
`subscribe:twin`	`{ twinId }`	Join twin room
`subscribe:sensor`	`{ sensorId }`	Join sensor room
`unsubscribe`	`{ room }`	Leave a room
`collaboration:join`	`{ twinId, user }`	Join collaboration room for a twin **(v0.4.0)**
`collaboration:leave`	`{ twinId }`	Leave collaboration room **(v0.4.0)**
`collaboration:select`	`{ twinId, meshName, userId }`	Broadcast mesh selection to peers **(v0.4.0)**
`collaboration:sensor-edit`	`{ twinId, sensorId, changes, userId }`	Broadcast sensor edit (last-write-wins) **(v0.4.0)**

Server Events (listen):

Event	Payload	Description
`sensor:data`	`{ sensorId, tenantId, value, timestamp, mode, metadata }`	Real-time sensor reading
`alert`	`{ id, severity, message, value, sensorId, tenantId, createdAt }`	Threshold alert (new in v0.2.0)
`collaboration:presence`	`{ users: CollaborationUser[] }`	Updated list of users in the collaboration room **(v0.4.0)**
`collaboration:selection`	`{ userId, meshName }`	Peer's mesh selection **(v0.4.0)**
`collaboration:sensor-edited`	`{ userId, sensorId, changes }`	Peer's sensor edit **(v0.4.0)**

Throttling (v0.2.0): WebSocket broadcasts are throttled to max 10 updates/second per room. The latest event per room is buffered and flushed every 100ms, shedding intermediate updates under load.

Deduplication: Redis subscriptions are created once per unique subscription key, preventing duplicate listeners when multiple clients subscribe to the same tenant/sensor.

Reconnection: Subscriptions are tracked in `pendingSubscriptions` and re-emitted on reconnect.

Collaboration (v0.4.0): The RealtimeGateway tracks active users per twin. On join, it emits the full presence list to all users in the room. On disconnect, it automatically cleans up presence. Selection and sensor-edit events are broadcast to all peers except the sender.

5.6 Sensor Streaming Engine

The `SensorStreamService` generates simulated sensor data:

Pattern	Description
`CONSTANT`	Fixed value at `(min + max) / 2`
`LINEAR_INCREASE`	Linearly increases from min to max, then wraps
`LINEAR_DECREASE`	Linearly decreases from max to min, then wraps
`SINE`	Sine wave oscillating between min and max
`RANDOM`	Random values between min and max

Each stream runs on a configurable interval (default 1000ms). On each tick:

1. Generates value based on pattern
2. Writes to TimescaleDB via `TimescaleService.insertSensorData()`
3. Publishes to Redis channel `sensor:{sensorId}:data`
4. The realtime gateway picks up the Redis message and broadcasts via WebSocket

v0.2.0 changes:

- Starting a stream sets sensor mode to `STREAM` and clears any manual override
- Stopping a stream reverts mode to `REAL` and sets `streamActive=false`
- Duplicate streams per sensor are prevented (returns existing stream info)
- All intervals are cleaned up on service destroy (`onModuleDestroy`)
- Setting a manual override automatically stops any active stream first

Single Source of Truth: Only one data source is active at a time:

- `REAL` mode — accepts external ingestion (HTTP, MQTT)
- `MANUAL` mode — rejects external ingestion, value set via override
- `STREAM` mode — rejects external ingestion, value generated by stream engine

5.7 Data Ingestion

Two ingestion paths:

HTTP Ingestion:

- `POST /api/v1/ingest` — single data point
- `POST /api/v1/ingest/batch` — batch of data points (parallel processing, chunks of 10)
- Validates sensor existence, checks sensor mode, writes to TimescaleDB, publishes to Redis
- **v0.2.0:** Returns accept/reject status with reason. Rate limited to 20 requests/sec per sensor.
- **v0.2.0:** `GET /ingest/metrics` endpoint exposes totalIngested, totalRejected, totalRateLimited counters

MQTT Ingestion:

- Connects to Mosquitto broker at startup
- Subscribes to `sensors/+/data` wildcard topic
- Parses JSON payload, extracts sensorId from topic
- Routes through the same ingestion pipeline as HTTP

5.8 Analytics

Three query types powered by TimescaleDB:

Query	SQL Feature	Description
History	`ORDER BY time DESC LIMIT N`	Raw time-series data for a sensor
Latest	`ORDER BY time DESC LIMIT 1`	Most recent reading
Aggregated	`time_bucket()`	Averaged/min/max/count per time bucket

Analytics responses are cached in Redis with configurable TTL.

5.9 Alert System

New in v0.2.0, enhanced in v0.4.0. The alert system monitors sensor data against configurable thresholds and AI anomaly detection.

How it works:

1. Each sensor can have `alertMinThreshold` and `alertMaxThreshold` fields (nullable)
2. On every data ingestion (including stream ticks), the value is checked against thresholds
3. **Hysteresis (v0.4.0):** Value must exceed threshold by `alertHysteresis` amount before triggering
4. **Cooldown (v0.4.0):** Alerts are suppressed for `alertCooldownMs` milliseconds after the last alert for the same sensor (default 30s)
5. If threshold is breached (after hysteresis), a `WARNING` or `CRITICAL` alert is created
6. The alert is published to Redis channel `alert:{tenantId}`
7. `RealtimeService` picks up the Redis message and broadcasts via WebSocket `alert` event
8. Frontend toast notification and AlertsPage update in real-time

Anomaly Detection Alerts (v0.4.0):

1. Every ingested data point is also analyzed by `AnomalyDetectionService`
2. Rolling statistical model calculates z-score and spike detection → anomaly score (0–100)
3. If anomaly detected: score < 80 → `INFO` alert, score ≥ 80 → `WARNING` alert
4. Anomaly alerts have a separate 60-second cooldown per sensor

Alert acknowledgment: Alerts can be acknowledged individually or in bulk. Only `ADMIN` and `MANAGER` can acknowledge all.

5.10 Health & Observability

New in v0.2.0. Three health endpoints for Kubernetes/Docker health probes:

Endpoint	Purpose	Checks
`GET /health`	Full system health	Database, Redis, TimescaleDB (with latency)
`GET /health/live`	Liveness probe	Always returns `{ status: "ok" }`
`GET /health/ready`	Readiness probe	Database connectivity

These endpoints are **unauthenticated** for use by container orchestrators.

5.11 3D Model Management

New in v0.3.0, enhanced in v0.4.0. The models module provides full lifecycle management for 3D assets including versioning and soft delete.

Upload Pipeline:

```
User uploads a supported model file (.glb, .gltf, or .obj) via POST /models (multipart/form-data)
  → ModelsController validates file type and size
  → ModelsService.uploadModel()
    → Validates twin belongs to tenant
    → Saves file to ./uploads/models/{uuid}.{ext}
    → Parses GLTF/GLB to extract mesh names
    → Creates Model3D + ModelPart records in transaction
    → Returns model with parts (each part has UUID for binding)
```

Supported Formats:

Format	Extension	Mesh Parsing
glTF Binary	`.glb`	Yes — extracts mesh names from binary JSON chunk
glTF	`.gltf`	Yes — parses JSON to extract mesh node names
Wavefront OBJ	`.obj`	Yes — loads geometry as a single part when mesh names are unavailable

> FBX uploads are no longer supported. Use GLB, GLTF, or OBJ files for reliable viewer rendering.

File Storage:

- Default: local disk at `./uploads/models/`
- CDN-ready: set `CDN_BASE_URL` env var to prefix all file URLs
- Static file serving via `app.useStaticAssets()` at `/uploads`
- Frontend dev servers must proxy `/uploads` to the backend so model requests return the binary file instead of the Vite HTML shell

Metadata Cache:

- In-memory cache with 60-second TTL for model list and detail queries
- Automatically invalidated on upload, update, and delete operations

Sensor Binding by Model Part UUID:

- Each mesh in an uploaded model gets a `ModelPart` record with a UUID
- Sensors bind to `modelPartId` (not mesh name), ensuring stable references
- `bindToModelPart` validates that the model part belongs to the tenant
- On model delete, all bound sensors are unbound before deletion

Database Indexes:

- `@@index([twinId])` — fast queries by twin
- `@@index([tenantId, twinId])` — fast tenant-scoped queries

Model Versioning (v0.4.0):

- Each model has a `version` number (starts at 1) and `isLatest` flag
- `uploadNewVersion(modelId, file)` creates a child record with `parentModelId` pointing to the root, increments version, and demotes the previous latest
- `getVersionHistory(modelId)` returns all versions in the lineage ordered by version descending
- `rollbackToVersion(modelId)` promotes the target version to `isLatest` and demotes all others
- Version lineage is preserved — rollback doesn't delete, it re-promotes

Soft Delete (v0.4.0):

- `DELETE /models/:id` sets `deletedAt` instead of destroying the record
- All list/get queries filter out deleted records by default
- `?includeDeleted=true` query param to include soft-deleted records
- `POST /models/:id/restore` clears `deletedAt` to restore
- `DELETE /models/:id/permanent` (ADMIN only) truly deletes the record, file, and parts

5.12 AI Anomaly Detection

New in v0.4.0. The `AnomalyDetectionService` provides real-time statistical anomaly detection on sensor data.

Algorithm:

```
For each sensor data point:
  1. Add value to rolling window (last 100 values)
  2. Calculate rolling mean and standard deviation
  3. Z-score = |value - mean| / stddev  →  if > 3σ, flag as anomaly
  4. Spike detection = |value - previousValue| / mean  →  if > 50%, flag as spike
  5. Anomaly score = 0.6 * zScoreComponent + 0.4 * spikeComponent  (0–100)
  6. If score > 50 → anomaly detected → create alert (INFO or WARNING based on score)
```

Bootstrap: `bootstrapSensor(sensorId)` loads the last 24 hours of historical data to pre-fill the rolling window, enabling immediate detection on the first new data point.

Integration: Imported into `SensorsModule` and called automatically on every `recordData()` invocation.

5.13 Device Validation Layer

New in v0.4.0. Per-sensor schema validation for IoT device data.

- Each sensor can have a `validationSchema` JSON field
- Supported validation rules: `minValue`, `maxValue`, `valueType` (e.g. "integer"), `requiredMetadata` (array of keys)
- `SensorsService.validatePayload()` is called before data ingestion
- Invalid payloads are rejected with descriptive error messages

---

6. Frontend (React)

6.1 Pages

Route	Page	Description
`/login`	`LoginPage`	Email/password login form + Google OAuth button. Pre-filled with demo credentials
`/`	`DashboardPage`	KPI cards (twins, sensors, active streams, **3D models**, alerts), area chart, live sensor values, alert panel
`/twins`	`TwinsPage`	Twin list with create/**edit/delete** form, asset hierarchy tree with **edit/delete actions (updated v0.3.0)**
`/viewer`	`ViewerPage`	3D viewer with model selector, **version selector**, **playback controls**, **collaboration presence**, **drag-and-drop sensor binding (updated v0.4.0)**
`/models`	`ModelsPage`	Upload, list, edit, **versioning**, **soft delete/restore**, **version selector**, **skeleton loading (updated v0.4.0)**
`/sensors`	`SensorsPage`	Sensor grid with **create/edit/delete** actions, live values, bound status **(updated v0.3.0)**
`/sensor-testing`	`SensorTestingPage`	**Core feature** — left: sensor tree, right: full control panel
`/alerts`	`AlertsPage`	Alert list with **INFO/WARNING/CRITICAL badges**, filter (all/unacknowledged), acknowledge, improved empty state **(updated v0.4.0)**

All pages except `/login` are **protected** — unauthenticated users are redirected to login. Pages are **lazy-loaded** via `React.lazy()` for performance.

6.2 Components

UI Components (`components/ui/`)

- **Button** — Variants: primary, secondary, destructive, outline, ghost. Sizes: sm, md, lg
- **Card** — Card container with CardHeader, CardTitle, CardContent sub-components
- **Input** — Styled text/number input with focus ring
- **Badge** — Status badges: default (blue), success (green), warning (yellow), danger (red)
- **ErrorBoundary** — React error boundary with retry button and error message **(v0.2.0)**
- **Toast** — Global toast notification system with success/error/warning/info types, auto-dismiss **(v0.2.0)**
- **ConfirmDialog** — Reusable confirmation modal for destructive actions (delete twin/asset/sensor/model) with optional warning message **(v0.3.0)**
- **ConnectionStatus** — WebSocket connection status indicator (connected/connecting/reconnecting/disconnected) **(v0.2.0)**
- **Skeleton** — Animated loading placeholders: `Skeleton` (base), `CardSkeleton`, `TableSkeleton`, `ListSkeleton` **(v0.4.0)**
- **EmptyState** — Empty data display with icon, title, description, and optional action button **(v0.4.0)**

Layout (`components/layout/`)

- **AppLayout** — Protected wrapper with sidebar + header (ConnectionStatus) + ErrorBoundary-wrapped content outlet **(updated v0.2.0)**
- **Sidebar** — Navigation with Lucide icons: Dashboard, Digital Twins, 3D Viewer, **3D Models**, Sensors, Testing Panel, Alerts, Logout **(updated v0.3.0)**

3D Components (`components/3d/`)

- **SceneViewer** — Three.js canvas wrapped in `SceneDragDrop` for drag-and-drop binding. `SceneCapture` inner component exposes camera/scene/canvas refs for raycasting. Builds meshName→modelPartId map from active model **(updated v0.4.0)**
- **DemoFactory** — Interactive 3D factory floor with:
  - **Motor unit** — clickable, bound to temperature sensor
  - **Conveyor belt** — clickable, bound to vibration sensor
  - **Hydraulic press** — clickable, bound to pressure sensor
  - **Robotic arm** — clickable, bound to RPM sensor
  - **Control panel** — decorative element with green screen
  - **Color coding:** green (normal, <60), yellow (warning, 60-80), red (critical, >80)
  - **Selection highlight:** purple glow when clicked
- **UploadedModel** — Dynamic 3D model loader using GLTFLoader/OBJLoader. Auto-centers and scales models. Maps mesh names to ModelPart UUIDs. Color-codes meshes by sensor value (green/yellow/red). Highlights selected mesh **(v0.3.0)**
- **SensorDragOverlay** — `SceneDragDrop`: HTML wrapper around canvas that handles dragOver/drop events; raycasts into the Three.js scene to find the mesh under cursor; highlights target mesh with blue emissive glow; on drop calls `sensorsApi.bind()`. `DraggableSensorChip`: draggable chip that puts `sensorId` into `dataTransfer` **(v0.4.0)**

Sensor Components (`components/sensors/`)

- **SensorBindingPanel** — Appears when a mesh/part is selected. Shows bound sensors with live values and unbind button. Bind Existing Sensor dropdown. **Drag to 3D Model** section with `DraggableSensorChip` for each unbound sensor. Create & Bind form **(updated v0.4.0)**

Model Components (`components/models/`)

- **VersionSelector** — Dropdown showing version history for a model. Click a version to load it; rollback button to promote an older version to latest. Fetches from `modelsApi.getVersions()` **(v0.4.0)**

Playback Components (`components/playback/`)

- **PlaybackControls** — Sensor history playback toolbar: time-range selector (1h/6h/24h/7d), play/pause/skip buttons, speed control (0.5x–5x), progress bar with frame counter. Loads data from `analyticsApi.history()` **(v0.4.0)**

Collaboration Components (`components/collaboration/`)

- **PresenceIndicator** — Displays active collaborators as colored avatar circles with user initials. Current user is marked with a ring. Shows count badge if > 3 users **(v0.4.0)**

6.3 State Management

Four **Zustand** stores:

`authStore`

Field	Type	Description
`user`	`User | null`	Current authenticated user
`isAuthenticated`	`boolean`	Auth status
`isLoading`	`boolean`	Loading state
`login()`	function	Login with email/password, store tokens
`register()`	function	Register new user
`logout()`	function	Clear tokens, reset state

`twinStore`

Field	Type	Description
`twins`	`DigitalTwin[]`	All twins
`currentTwin`	`DigitalTwin | null`	Selected twin
`assets`	`Asset[]`	Assets of current twin
`fetchTwins()`	function	Load all twins from API
`selectTwin()`	function	Select twin + load assets
`createTwin()`	function	Create new twin

`sensorStore`

Field	Type	Description
`sensors`	`Sensor[]`	All sensors
`selectedSensor`	`Sensor | null`	Currently selected sensor
`realtimeValues`	`Map<string, SensorDataPoint>`	Live values by sensor ID
`overrideSensor()`	function	Set manual override
`clearOverride()`	function	Return to REAL mode
`startStream()`	function	Start simulated data stream
`stopStream()`	function	Stop stream
`initWebSocket()`	function	Connect WebSocket + subscribe to all sensor events

`viewerStore`

Field	Type	Description
`selectedMeshName`	`string | null`	Name of clicked mesh
`selectedMeshId`	`string | null`	Associated ID
`selectedModelPartId`	`string | null`	UUID of selected model part **(v0.3.0)**
`hoveredMeshName`	`string | null`	Currently hovered mesh
`activeModelId`	`string | null`	Currently loaded model ID **(v0.3.0)**
`selectMesh()`	function	Set selected mesh (name, id, modelPartId)
`clearSelection()`	function	Deselect
`setActiveModel()`	function	Set active model for viewer **(v0.3.0)**

6.4 Services

API Service (`services/api.ts`)

- Axios instance with base URL `/api/v1`
- Request interceptor: attaches `Authorization: Bearer <token>` header
- Response interceptor: on 401, clears tokens and redirects to `/login`
- Organized into namespaces: `authApi`, `twinsApi`, `assetsApi`, `sensorsApi`, `ingestApi`, `analyticsApi`, `alertsApi`, `healthApi`, `modelsApi`, `exportApi` **(v0.4.0)**
- **v0.3.0:** `modelsApi` — list, get, upload, update, delete, boundSensors
- **v0.4.0:** `modelsApi` extended — `uploadVersion()`, `getVersions()`, `rollback()`, `restore()`, `permanentDelete()`
- **v0.4.0:** `exportApi` — `sensorCsv()`, `twinJson()`, `createShareLink()`, `getShared()`, `revokeShare()`
- **v0.2.0:** Automatic retry on 5xx errors for GET requests (1 retry, 1s delay)

WebSocket Service (`services/websocket.ts`)

- Socket.IO client connecting to `/realtime` namespace
- Methods: `connect()`, `disconnect()`, `subscribeTenant()`, `subscribeSensor()`, `subscribeTwin()`
- Listener system: `onSensorData(sensorId, callback)` — returns unsubscribe function
- `onAllSensorData(callback)` — wildcard listener for all sensor events
- **v0.2.0 additions:**
  - `onAlert(callback)` — listen for alert events
  - `onStatusChange(callback)` — track connection status changes
  - Auto-reconnect with exponential backoff (1s–10s)
  - Pending subscriptions replayed on reconnect
  - Connection status tracking: `connected`, `connecting`, `reconnecting`, `disconnected`
- **v0.4.0 additions:**
  - `joinCollaboration(twinId, user)` — join collaboration room
  - `leaveCollaboration(twinId)` — leave collaboration room
  - `broadcastSelection(twinId, meshName)` — share mesh selection with peers
  - `broadcastSensorEdit(twinId, sensorId, changes)` — share sensor edit with peers
  - `onPresence(callback)`, `onPeerSelection(callback)`, `onPeerSensorEdit(callback)` — listeners

---

7. Infrastructure & DevOps

7.1 Docker Compose Services

Service	Image	Port (host:container)	Purpose
`app`	Built from Dockerfile	3000:3000	NestJS backend
`postgres`	timescale/timescaledb:latest-pg16	5433:5432	PostgreSQL + TimescaleDB
`redis`	redis:7-alpine	6379:6379	Caching + pub/sub
`kafka`	apache/kafka:3.7.0	9092:9092	Event streaming (KRaft mode, no Zookeeper)
`mosquitto`	eclipse-mosquitto:2	1884:1883, 9002:9001	MQTT broker
`nginx`	nginx:alpine	80:80	API gateway / reverse proxy

Volumes: pgdata, redisdata, mosquittodata, mosquittolog, uploads

Network: `nebula-net` (bridge)

Health checks:

- PostgreSQL: `pg_isready -U nebulaTwin -d nebula_twin`
- Redis: `redis-cli -a nebulaTwin ping`

7.2 Dockerfile

Multi-stage build:

1. **Builder stage** — `node:20-alpine`, installs deps, generates Prisma client, builds NestJS
2. **Production stage** — `node:20-alpine`, copies dist + node_modules + prisma, runs as non-root user `nestjs:nodejs`

Health check: `wget --spider http://localhost:3000/api/v1/health`

7.3 NGINX API Gateway

Configuration features:

- Rate limiting: 10 req/s per IP (burst 20)
- Security headers: X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy
- Proxy: `/api/` → backend:3000, `/docs` → backend:3000
- WebSocket: `/realtime` → backend:3000 (with upgrade headers)
- Health check: `/health` returns 200

---

8. Environment Configuration

Copy `.env.example` to `.env` and configure:

Variable	Default	Description
`NODE_ENV`	development	Environment mode
`PORT`	3000	Backend server port
`API_PREFIX`	api/v1	API route prefix
`DATABASE_URL`	—	PostgreSQL connection string
`REDIS_HOST`	localhost	Redis hostname
`REDIS_PORT`	6379	Redis port
`REDIS_PASSWORD`	—	Redis password
`KAFKA_BROKERS`	localhost:9092	Kafka broker addresses
`KAFKA_CLIENT_ID`	nebula-twin	Kafka client identifier
`KAFKA_GROUP_ID`	nebula-twin-group	Kafka consumer group
`JWT_SECRET`	—	JWT signing secret
`JWT_EXPIRES_IN`	1d	Access token expiry
`JWT_REFRESH_SECRET`	—	Refresh token signing secret
`JWT_REFRESH_EXPIRES_IN`	7d	Refresh token expiry
`GOOGLE_CLIENT_ID`	—	Google OAuth client ID (optional)
`GOOGLE_CLIENT_SECRET`	—	Google OAuth client secret (optional)
`GOOGLE_CALLBACK_URL`	http://localhost:3000/api/v1/auth/google/callback	OAuth callback URL
`MQTT_URL`	mqtt://localhost:1883	MQTT broker URL
`MQTT_USERNAME`	—	MQTT username
`MQTT_PASSWORD`	—	MQTT password
`UPLOAD_DIR`	./uploads	File upload directory
`MAX_FILE_SIZE`	104857600	Max upload size (100MB)
`CDN_BASE_URL`	*(empty)*	CDN URL prefix for model file URLs (e.g. `https://cdn.example.com`). If empty, uses local `/uploads` path **(v0.3.0)**
`THROTTLE_TTL`	60	Rate limit window (seconds)
`THROTTLE_LIMIT`	100	Max requests per window
`CORS_ORIGINS`	http://localhost:3000,http://localhost:4200	Allowed CORS origins

9. Setup & Running

Prerequisites

- Node.js 20+
- Docker & Docker Compose
- PostgreSQL client (`psql`) — optional, for manual DB access

Step-by-Step

```
# 1. Clone and enter project
cd nebula

# 2. Copy environment file
cp .env.example .env
# Edit .env with your credentials

# 3. Install backend dependencies
npm install

# 4. Start infrastructure containers
docker-compose up -d postgres redis kafka mosquitto

# 5. Wait for PostgreSQL to be healthy, then run migrations
npx prisma migrate dev --name init

# 6. Create the TimescaleDB hypertable (if not auto-created)
PGPASSWORD=<your_password> psql -h localhost -p 5433 -U <your_user> -d nebula_twin \
  -f prisma/migrations/00000000000000_init_timescaledb/migration.sql

# 7. Seed demo data
npx ts-node prisma/seed.ts

# 8. Start backend (dev mode with hot-reload)
npm run start:dev

# 9. Install frontend dependencies
cd frontend
npm install

# 10. Start frontend (dev mode with HMR)
npx vite --host

# Backend: http://localhost:3000
# Frontend: http://localhost:5173
# Swagger:  http://localhost:3000/docs
```

NPM Scripts (Backend)

Script	Command	Description
`start:dev`	`nest start --watch`	Development with hot-reload
`start:prod`	`node dist/main`	Production mode
`build`	`nest build`	Compile TypeScript
`prisma:migrate`	`prisma migrate dev`	Run database migrations
`prisma:seed`	`ts-node prisma/seed.ts`	Seed database
`docker:up`	`docker-compose up -d`	Start all containers
`docker:down`	`docker-compose down`	Stop all containers
`test`	`jest`	Run unit tests
`lint`	`eslint --fix`	Lint and auto-fix

---

10. Seeded Demo Data

The seed script (`prisma/seed.ts`) creates:

Entity	Details
Tenant	"NebulaTwin Demo Org" (`default-tenant-id`)
Admin User	`admin@nebulatwin.io` / `admin123` (role: ADMIN)
Digital Twin	"Smart Factory Alpha" (`demo-twin-id`)
Assets (hierarchical)	Factory → Production Line A → CNC Machine 01 → Spindle Motor
Sensors (4)	Temperature (°C), Vibration (mm/s), Pressure (bar), RPM (rpm)

Asset Hierarchy

```
Smart Factory Alpha
└── Main Factory Building (FACTORY)
    └── Production Line A (LINE)
        └── CNC Machine 01 (MACHINE)
            └── Spindle Motor (COMPONENT)
                ├── Temperature Sensor (°C)
                ├── Vibration Sensor (mm/s)
                ├── Pressure Sensor (bar)
                └── RPM Sensor (rpm)
```

Default Login

- **Email:** `admin@nebulatwin.io`
- **Password:** `admin123`

---

11. Data Flow Diagrams

Sensor Override Flow

```
User clicks "Apply" in Testing Panel
  → POST /api/v1/sensors/:id/override { mode: "manual", value: 72.5 }
  → SensorsService.setOverride()
    → Prisma UPDATE sensor (mode=MANUAL, manualValue=72.5)
    → TimescaleService.insertSensorData(sensorId, 72.5)
    → RedisService.publish("sensor:{id}:data", payload)
  → RealtimeService picks up Redis message
  → Socket.IO broadcasts "sensor:data" to subscribed clients
  → Frontend Zustand store updates realtimeValues Map
  → React re-renders:
    → 3D model mesh color changes (green/yellow/red)
    → Chart updates with new data point
    → Testing panel shows updated value
```

Sensor Stream Flow

```
User clicks "Start Stream" (pattern: SINE, interval: 1000ms, min: 10, max: 90)
  → POST /api/v1/sensors/:id/stream
  → SensorStreamService.startStream()
    → Creates setInterval(1000ms)
    → Every tick:
      → Calculate value based on SINE pattern
      → TimescaleService.insertSensorData()
      → RedisService.publish()
      → Socket.IO broadcasts to clients
  → User clicks "Stop Stream"
    → POST /api/v1/sensors/:id/stop
    → clearInterval(), update DB (streamActive=false)
```

MQTT Ingestion Flow

```
IoT Device publishes to: sensors/sensor-temp-01/data
  → Mosquitto broker receives message
  → MqttIngestionService.handleMessage()
    → Parse topic → extract sensorId
    → Parse JSON payload → extract value
    → IngestionService.ingestSingle()
      → TimescaleService.insertSensorData()
      → RedisService.publish()
      → Socket.IO broadcasts to clients
```

Drag & Drop Sensor Binding Flow (v0.4.0)

```
User drags DraggableSensorChip from sidebar/tray
  → HTML5 dragStart: e.dataTransfer.setData('sensorId', sensorId)
  → Mouse enters SceneDragDrop overlay (wrapping 3D canvas)
  → dragOver (throttled):
    → SceneDragDrop.raycastAtEvent(e)
      → Convert mouse coords to NDC using canvas bounds
      → THREE.Raycaster.setFromCamera(ndc, camera)
      → raycaster.intersectObjects(scene.children, true)
      → First hit mesh → set emissive blue highlight
      → Lookup mesh.name in partMap → get modelPartId
  → User drops on highlighted mesh:
    → Read sensorId from e.dataTransfer
    → POST /api/v1/sensors/:sensorId/bind/:modelPartId
    → Clear highlight, refresh sensorStore
    → Toast: "Sensor bound to {meshName}"
```

Anomaly Detection Flow (v0.4.0)

```
Sensor data ingested (HTTP, MQTT, or stream tick)
  → SensorsService.recordData(sensorId, value)
    → TimescaleService.insertSensorData()
    → AnomalyDetectionService.analyze(sensorId, value)
      → Add to rolling window (last 100 values)
      → Calculate z-score: |value - mean| / stddev
      → Calculate spike: |value - prev| / mean
      → Anomaly score = 0.6 * zScore + 0.4 * spike (0–100)
      → If score > 50 AND cooldown elapsed:
        → Create alert (INFO if score < 80, WARNING if ≥ 80)
        → Publish alert to Redis → broadcast via WebSocket
    → Check threshold alerts (with hysteresis + cooldown)
    → Publish sensor:data to Redis
```

---

12. Security

Feature	Implementation
Authentication	JWT (access + refresh tokens), Google OAuth 2.0
Password hashing	bcrypt (10 salt rounds)
RBAC	4 roles with guard-based enforcement
Tenant isolation	TenantGuard ensures cross-tenant data access is blocked
HTTP security headers	Helmet.js (X-Frame-Options, CSP, HSTS, etc.)
Rate limiting	NestJS Throttler (100 req/60s default) + NGINX (10 req/s)
Input validation	class-validator with whitelist + forbidNonWhitelisted
CORS	Configurable allowed origins
Non-root container	Docker runs as `nestjs` user (UID 1001)
Token storage	localStorage (frontend) — for production, use httpOnly cookies

---

13. Production Hardening (v0.2.0)

The following hardening changes were made in v0.2.0:

Backend

Area	Change
Sensor Modes	Added `STREAM` mode. Sensors now have 3 exclusive modes: `REAL`, `MANUAL`, `STREAM`
Single Source of Truth	Only one data source active at a time. Override stops streams; streams clear overrides
Ingestion Gating	External ingestion rejected when sensor is in `MANUAL` or `STREAM` mode
Rate Limiting	Per-sensor rate limit (20/sec) via Redis sliding window
Ingestion Metrics	`GET /ingest/metrics` endpoint for monitoring
Alert System	Configurable min/max thresholds per sensor. Alerts created on breach, broadcast via WebSocket
Health Endpoints	`/health`, `/health/live`, `/health/ready` for container orchestration
WebSocket Throttling	100ms flush interval with load shedding (latest-wins per room)
Redis psubscribe	Pattern-based subscriptions for sensor data and alert channels
Dedup Subscriptions	Redis subscriptions created once per unique key, not per client
RBAC Tightened	Override/stream endpoints restricted to ADMIN + MANAGER only
Override DTO	`mode` field deprecated and optional
Duplicate Prevention	`ON CONFLICT DO NOTHING` on TimescaleDB inserts
Error Handling	Try/catch with structured logging on TimescaleDB writes
Stream Cleanup	`onModuleDestroy` clears all intervals. Duplicate streams prevented
tsconfig	Excluded `frontend/` from backend TypeScript compilation

Frontend

Area	Change
ErrorBoundary	Wraps all routes — catches and displays errors with retry
Toast System	Global toast notifications for actions (success, error, warning, info)
ConnectionStatus	Header indicator showing WebSocket connection state
Auto-Reconnect	WebSocket reconnects with exponential backoff, replays subscriptions
API Retry	Auto-retries GET requests on 5xx errors (1 retry, 1s delay)
Alerts Page	Full alerts dashboard with filter, acknowledge, stats
Sidebar	Added Alerts navigation link
Types	Added `STREAM` mode, `AlertEvent`, `Alert`, `AlertSeverity`
Sensor Store	Toast notifications on all actions, error handling

Tests

Suite	Tests	Coverage
`sensors.service.spec.ts`	8	Mode enforcement, override flow, alert thresholds
`ingestion.service.spec.ts`	6	Validation, rate limiting, metrics, batch processing
`alerts.service.spec.ts`	5	CRUD, acknowledgment, stats
Total	**19**	All passing

---

14. Testing

Running Tests

```
# Run all tests
npm test

# Run specific suite
npx jest --testPathPattern="sensors.service.spec"

# Run with coverage
npm run test:cov
```

Test Architecture

All tests use **NestJS Testing Module** with mocked dependencies:

- `PrismaService` — mocked with `jest.fn()` per method
- `TimescaleService` — mocked for insert operations
- `EventBusService` — mocked for publish operations
- `RedisService` — mocked for rate limiting

Key Test Scenarios

Sensor Mode Enforcement:

- REAL mode accepts ingestion ✓
- MANUAL mode rejects ingestion ✓
- STREAM mode rejects ingestion ✓
- Missing sensor returns not_found ✓

Override Flow:

- Override stops active stream ✓
- Override without active stream skips stop ✓

Alert Thresholds:

- Value above max creates WARNING alert ✓
- Value within thresholds creates no alert ✓

Ingestion:

- Valid payload accepted ✓
- Invalid sensor_id rejected ✓
- NaN value rejected ✓
- Rate limit triggers at threshold ✓
- Batch with mixed results ✓

---

15. 3D Model Management (v0.3.0)

The following changes were made in v0.3.0:

Backend

Area	Change
Models Module	New `ModelsModule` with controller, service, DTOs for full 3D model lifecycle
File Upload	Multer-based multipart upload with file type/size validation. Supports `.glb`, `.gltf`, `.obj`
Mesh Parsing	Automatic extraction of mesh names from GLTF/GLB files, creating `ModelPart` records with UUIDs
Metadata Cache	In-memory cache (60s TTL) for model list/detail queries, auto-invalidated on mutations
CDN Support	`CDN_BASE_URL` env var prefixes file URLs for CDN delivery
Static Serving	`app.useStaticAssets()` serves `/uploads` directory
Cascade Delete	Model delete unbinds all sensors, removes file from disk, deletes model + parts
Sensor Binding	`bindToModelPart` validates model part exists and belongs to tenant
Schema	`Model3D` extended with `name`, `description`, `tenantId`, `ModelFormat` enum. Added `@@index([tenantId, twinId])`
Swagger	Added `models` tag to Swagger documentation

Frontend

Area	Change
ModelsPage	New page: drag & drop upload, file picker, twin selector, model card grid, edit modal, delete with bound-sensor warning
UploadedModel	New component: dynamic 3D model loader (GLTFLoader/OBJLoader), auto-center/scale, mesh-to-UUID mapping, realtime sensor color coding
SceneViewer	Accepts optional `model` prop — renders uploaded model or falls back to DemoFactory
ViewerPage	Model selector dropdown, URL param support (`?modelId=`), fetches models list
SensorBindingPanel	UUID-based binding, bind/unbind existing sensors, create & bind new sensors, displays `modelPartId`
TwinsPage	Added edit/delete for twins and assets with modal dialogs and confirmation
SensorsPage	Added create/edit/delete sensors with modals, threshold editing, bound status badge
DashboardPage	Added 3D Models KPI card, 5-column grid layout
Sidebar	Added “3D Models” navigation link (FileBox icon)
ConfirmDialog	New reusable confirmation modal for destructive actions
useRole Hook	Frontend RBAC utility: `canEdit`, `canDelete`, `canUpload`, `isAdmin`, `isViewer`
RBAC Enforcement	Upload/edit/delete buttons hidden for unauthorized roles (VIEWER=read-only, OPERATOR=no delete, MANAGER+=full CRUD)
viewerStore	Added `selectedModelPartId`, `activeModelId`, `setActiveModel()`
Types	Extended `Model3D`, `ModelPart` interfaces, added `ModelFormat` type
API Service	Added `modelsApi` namespace (list, get, upload, update, delete, boundSensors)

RBAC Summary (v0.3.0)

Action	ADMIN	MANAGER	OPERATOR	VIEWER
View models / twins / sensors	✓	✓	✓	✓
Upload / edit models	✓	✓	✗	✗
Delete models	✓	✓	✗	✗
Create / edit sensors	✓	✓	✓	✗
Delete sensors	✓	✓	✗	✗
Edit twins / assets	✓	✓	✓	✗
Delete twins / assets	✓	✓	✗	✗
Override / stream control	✓	✓	✗	✗
Permanent delete models **(v0.4.0)**	✓	✗	✗	✗
Create / revoke share links **(v0.4.0)**	✓	✓	✗	✗
Export CSV / JSON **(v0.4.0)**	✓	✓	✓	✓

---

16. v0.4.0 — Intelligence, Collaboration & Safety

16.1 3D Model Versioning

- **Schema**: Added `version`, `isLatest`, `parentModelId` fields to `Model3D`
- **Upload new version**: `POST /api/v1/models/:id/version` — creates a child version, demotes previous latest
- **Version history**: `GET /api/v1/models/:id/versions` — returns all versions in lineage
- **Rollback**: `POST /api/v1/models/:id/rollback` — promotes target version to latest
- **Frontend**: `VersionSelector` component with dropdown, rollback button; `UploadVersionModal` for new version upload; version badge on model cards

16.2 Soft Delete

- **Schema**: Added `deletedAt` field to `Model3D`
- `DELETE /api/v1/models/:id` now sets `deletedAt` (soft delete)
- `POST /api/v1/models/:id/restore` — restores soft-deleted model
- `DELETE /api/v1/models/:id/permanent` — permanently deletes (ADMIN only)
- All list/get queries filter out deleted records by default; `?includeDeleted=true` to include
- **Frontend**: “Show deleted” toggle, restore/permanent-delete buttons, “deleted” badge

16.3 Drag & Drop Sensor Placement

Architecture: HTML5 drag events on an overlay div + Three.js raycasting to resolve the mesh under the cursor.

Components:

- **`SceneDragDrop`** (`SensorDragOverlay.tsx`) — wraps the 3D canvas div; intercepts `dragOver` / `dragLeave` / `drop`
  - On `dragOver` : raycasts from mouse position through the Three.js scene → finds mesh → highlights it with blue emissive glow → stores `meshName` + `modelPartId`
  - On `drop` : reads `sensorId` from `dataTransfer` , resolves `modelPartId` from the highlighted mesh, calls `sensorsApi.bind(sensorId, modelPartId)` , refreshes sensor store, shows toast
  - Visual overlay shows “Drop sensor on highlighted mesh” during drag
- **`SceneCapture`** (`SceneViewer.tsx`) — tiny R3F component inside `<Canvas>` that writes live `camera` , `scene` , and `gl.domElement` into a shared ref (exposed as `SceneInternals` interface)
- **`SceneViewer`** — builds `partMap` (meshName → modelPartId) from the active model, wraps `<Canvas>` in `<SceneDragDrop>` , passes ref + partMap
- **`DraggableSensorChip`** (`SensorDragOverlay.tsx`) — draggable chip that sets `sensorId` in `dataTransfer` on drag start

Drag sources (unbound sensors):

- `SensorBindingPanel` — “Drag to 3D Model” section with chips for each unbound sensor (visible when a mesh is selected)
- `ViewerPage` — always-visible floating sensor tray when unbound sensors exist and no mesh is selected

Flow:

```
User drags DraggableSensorChip from sidebar
  → dragStart: e.dataTransfer.setData('sensorId', id)
  → dragOver: SceneDragDrop.raycastAtEvent()
    → Raycaster casts into scene from mouse NDC coords
    → First intersected mesh → highlight with emissive blue
    → Resolve meshName → modelPartId via partMap
  → drop: read sensorId from dataTransfer
    → sensorsApi.bind(sensorId, modelPartId)
    → fetchSensors() to refresh store
    → toast success/error
```

16.4 Real-Time Collaboration

- **Presence system**: WebSocket events `collaboration:join` , `collaboration:leave` , `collaboration:presence`
- **Selection broadcast**: `collaboration:select` — broadcasts mesh selection to other users
- **Sensor edit broadcast**: `collaboration:sensor-edit` — last-write-wins conflict resolution
- **Frontend**: `PresenceIndicator` component shows active collaborators with avatar colors
- Automatic cleanup on disconnect

16.5 AI Anomaly Detection

- **Module**: `AnomalyDetectionService` — rolling window statistical model
- **Algorithm**: Z-score (3σ) + sudden spike detection with configurable thresholds
- **Anomaly score**: 0-100 (60% z-score weight, 40% spike weight)
- **Integration**: Automatically analyzed on every sensor data ingest
- **Alerts**: Anomaly triggers INFO (score < 80) or WARNING (score ≥ 80) alerts with 60s cooldown
- **Bootstrap**: `bootstrapSensor()` loads last 24h of data on init for immediate detection

16.6 Sensor History Playback

- **Component**: `PlaybackControls` — time range selector, play/pause/skip, speed control (0.5x-5x)
- Loads historical data for all selected sensors and replays frame-by-frame
- Progress bar with frame counter
- Integrated into ViewerPage with toggle button

16.7 Export & Sharing

- **CSV export**: `GET /api/v1/export/sensors/:id/csv?from=&to=` — downloads sensor data as CSV
- **JSON export**: `GET /api/v1/export/twins/:id/json` — exports full twin configuration
- **Share links**: `POST /api/v1/export/twins/:id/share` — creates time-limited public read-only link
- **Public access**: `GET /api/v1/export/shared/:token` — no auth required
- **Revoke**: `DELETE /api/v1/export/shared/:token`

16.8 Device Validation Layer

- **Schema validation**: `validationSchema` JSON field on Sensor model
- Supports: `minValue`, `maxValue`, `valueType` (integer check), `requiredMetadata`
- `SensorsService.validatePayload()` rejects invalid payloads with descriptive errors

16.9 Advanced Alert System

- **INFO severity**: Added to `AlertSeverity` enum — used for anomaly detection alerts
- **Alert cooldown**: `alertCooldownMs` per sensor (default 30s) — prevents alert spam
- **Hysteresis**: `alertHysteresis` per sensor — value must exceed threshold by hysteresis amount
- **Frontend**: INFO severity badge (blue), improved empty state

16.10 UI/UX Refinement

- **Loading skeletons**: `Skeleton`, `CardSkeleton`, `TableSkeleton`, `ListSkeleton` components
- **Empty states**: `EmptyState` component with icon, title, description, and action
- **Navigation**: Version badges on model cards, playback toggle in viewer
- **Model cards**: Version badge, deleted badge, restore/permanent-delete actions

New API Endpoints (v0.4.0)

Method	Path	Description
POST	`/models/:id/version`	Upload new model version
GET	`/models/:id/versions`	Get version history
POST	`/models/:id/rollback`	Rollback to version
POST	`/models/:id/restore`	Restore soft-deleted model
DELETE	`/models/:id/permanent`	Permanently delete (ADMIN)
GET	`/export/sensors/:id/csv`	Export sensor CSV
GET	`/export/twins/:id/json`	Export twin JSON
POST	`/export/twins/:id/share`	Create share link
GET	`/export/shared/:token`	Access shared twin
DELETE	`/export/shared/:token`	Revoke share link

New WebSocket Events (v0.4.0)

Event	Direction	Description
`collaboration:join`	Client → Server	Join twin collaboration room
`collaboration:leave`	Client → Server	Leave collaboration room
`collaboration:presence`	Server → Client	Updated user presence list
`collaboration:select`	Client → Server	Broadcast mesh selection
`collaboration:selection`	Server → Client	Receive selection from peer
`collaboration:sensor-edit`	Client → Server	Broadcast sensor edit
`collaboration:sensor-edited`	Server → Client	Receive edit from peer

Schema Changes (v0.4.0)

Model3D — added: `version` (Int), `isLatest` (Boolean), `parentModelId` (String?), `deletedAt` (DateTime?)

Sensor — added: `alertCooldownMs` (Int?), `alertHysteresis` (Float?), `validationSchema` (Json?)

AlertSeverity — added: `INFO`

---

17. Troubleshooting

Port conflicts

```
# Check what's using a port
lsof -i :3000
lsof -i :5432

# Kill process on port
kill $(lsof -t -i:3000)
```

`sensor_data` table missing

If Prisma migrate resets the schema, the TimescaleDB hypertable may be dropped:

```
PGPASSWORD=<password> psql -h localhost -p 5433 -U <user> -d nebula_twin \
  -f prisma/migrations/00000000000000_init_timescaledb/migration.sql
```

Kafka image not found

Use `apache/kafka:3.7.0` instead of `bitnami/kafka` (Bitnami tags are unpredictable).

Google OAuth crash on startup

If `GOOGLE_CLIENT_ID` is not a valid OAuth client ID, the Google strategy is automatically skipped. Set real credentials or leave as placeholder — the app boots either way.

JSONB cast error on sensor data insert

The TimescaleDB metadata column is `JSONB`. Ensure the INSERT uses `$3::jsonb` cast.

Backend hot-reload not working

NestJS `--watch` mode may hold the old process. Kill and restart:

```
kill $(lsof -t -i:3000); npm run start:dev
```

3D model renders blank or as HTML

If the browser console logs `Unexpected line: "<!doctype html>"` or the viewer shows a blank panel, the model URL is being served by the frontend dev server instead of the backend. Make sure `frontend/vite.config.ts` proxies `/uploads` to `http://localhost:3000`, then reload the viewer.

NebulaTwin Pro v0.4.0 — Built with NestJS, React, Three.js, TimescaleDB, Redis, Kafka, and MQTT.